

Supporting Information

A Machine Learning Approach to Delineate Impact of Materials Properties on Solar Cell Device Physics

Md. Shafiqul Islam^{1§}, Md Tohidul Islam^{2§}, Saugata Sarker^{1§}, Hasan Al Jame¹, Sadiq Shahriyar Nishat³, Md. Rafsun Jani¹, Abrar Rauf¹, Sumaiyatul Ahsan¹, Kazi Md. Shorowordi¹, Harry Efstathiadis⁴, Joaquin Carbonara⁵, Saquib Ahmed^{6*}

¹ Department of Materials and Metallurgical Engineering (MME), Bangladesh University of Engineering and Technology (BUET), East Campus, Dhaka-1000, Bangladesh

² Department of Materials Design and Innovation, University at Buffalo, Buffalo, NY 14260, USA

³ Department of Materials Science and Engineering (MSE), Rensselaer Polytechnic Institute, 110 8th street, Troy, NY 12180, USA

⁴ College of Nanoscale Science and Nanoengineering, SUNY Polytechnic Institute, 257 Fuller Road, Albany, NY 12203, USA

⁵ Department of Mathematics, SUNY – Buffalo State, 1300 Elmwood Avenue, Buffalo, NY 14222, USA

⁶ Department of Mechanical Engineering Technology, SUNY – Buffalo State, 1300 Elmwood Avenue, Buffalo, NY 14222, USA

§ M. Shafiqul Islam, M. Tohidul Islam and S. Sarker contributed equally in production of this manuscript.

*Corresponding author

Email: ahmedsm@buffalostate.edu, Phone: 1.716.878.6006 Fax: 1.716.878.3033

Jupyter Notebook

The step-by-step Machine Learning workflow is included below. They are attached as generated pdf via LaTeX from the jupyter notebook file.

How to Use the Notebook

1. Using Anaconda: A Python Distribution

Anaconda is the most widely used Python distribution for data science and comes pre-loaded with all the most popular libraries and tools. Some of the biggest Python libraries included in Anaconda include NumPy, pandas, and Matplotlib etc. To get Anaconda, simply:

1. Download the latest version of Anaconda for Python 3.8.
2. Install Anaconda by following the instructions on the download page and/or in the executable.

3. Open Anaconda and install and launch Jupyter.
4. Redirects to the URL for the dashboard `https://localhost:8888/tree` in your browser, Localhost is not a website, but served from your *local* machine: your own computer. A screenshot of the Jupyter dashboard looks like **Figure S1**.



Figure S1: A screenshot of the Jupyter dashboard

5. Browse and open the (.ipynb) file.

2. Using Google Colaboratory

See below to know how to open the (.ipynb) file using Google Collaboratory and run it.

1. The app needed is "Google Colaboratory". To install it, first go to Google Drive
2. Right click anywhere on the blank region → click "More" → click "Connect more apps". Follow the screenshot shown in **Figure S2**.

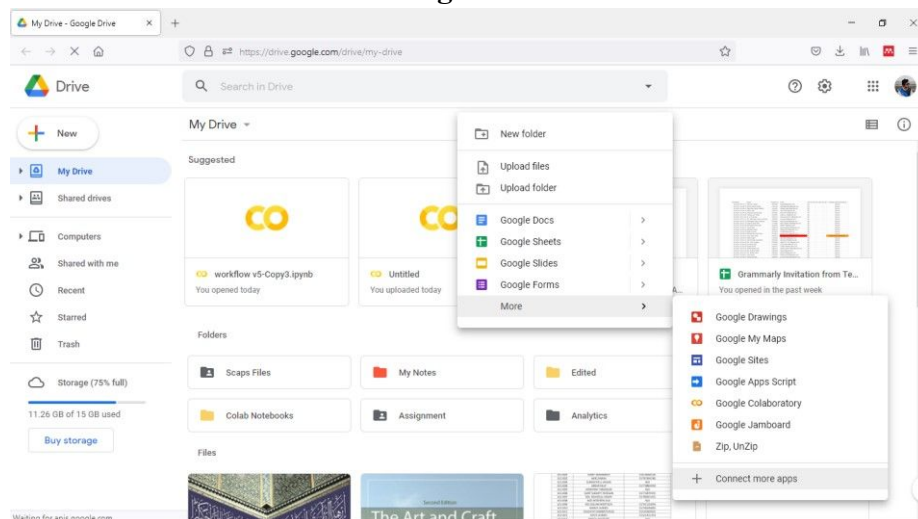


Figure S2: Installing Google Colaboratory from Google Drive

3. Then search for "Google Colaboratory" and install it. Follow the screenshot shown in **Figure S3**.

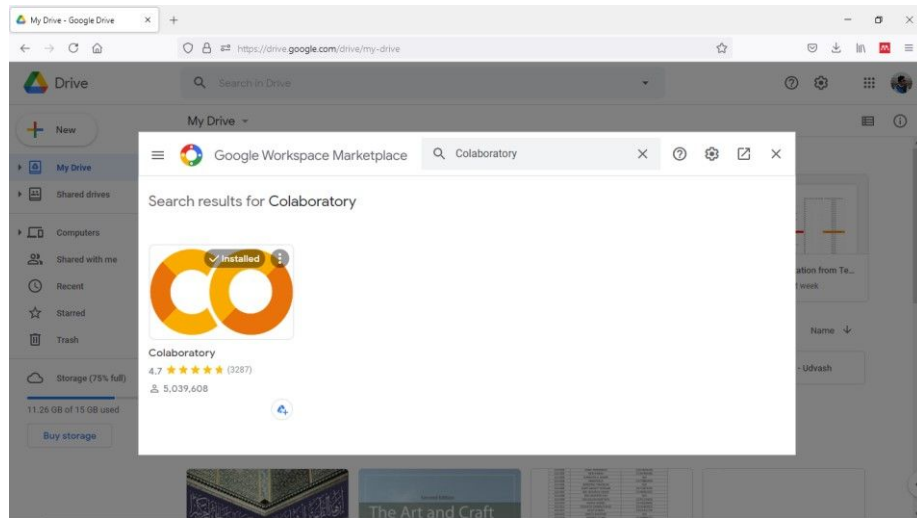


Figure S3: Installing Google Colaboratory

4. After installing "Google Colaboratory", go to colab.research.google.com, upload .ipynb file and run it. If the (.ipynb) file is in your google drive you can browse and then when you click on the (.ipynb) file, at the top you can choose "Open with Google Colab".

Machine Learning Workflow

Objective

We have the input parameters for absorber layer (Perovskite) : VB DoS, CB DoS, EA, Eg, e Mobility , h Mobility and Permittivity ranging as follows:

	VB DoS	CB DoS	EA	Eg	e Mobility	h Mobility	Permittivity
Ranges	2e18-1e19	2e18-1e19	4.15-4.30	1.5-1.7	20-100	20-100	7-11
Increments	2e18	2e18	0.05	0.05	20	20	1
Steps	5	5	4	5	5	5	5

We calculated Voc, Jsc, FF(%) and PCE(%) computationally using solar cell simulation software SCAPS, taking all the different combinations of our input parameters. So we got 63500 unique devices and their efficiency. We are going to apply machine learning models to our dataset to provide insight into relative strengths of solar device materials properties with a goal to make smart decisions in materials synthesis, processing and device fabrication methodologies and conditions.

Python Packages

There are a number of Python packages we'll use. Each of the import statements below imports or loads these packages in so that we can use them. A brief description is given for each one.

```
In [1]: import os # OS stands for Operating System and provides basic system functions
from collections import Counter # Collections is a package for handling data structures
from pprint import pprint
import pandas as pd # Pandas is a data analysis library which provides high performance,
import numpy as np # Numpy is a package for scientific computing with arrays
```

```
In [2]: import matplotlib # Matplotlib is the plotting package that provides a
import matplotlib.pyplot as plt # interface to the plotting library
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns # Seaborn is a Python data visualization library
```

```
In [3]: import sklearn # Scikit-learn is a machine learning package
from sklearn import metrics
from sklearn import tree
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_validate, GridSearchCV, ParameterGrid
from sklearn.model_selection import KFold, RepeatedKFold
from sklearn.tree import export_graphviz
```

```
In [5]: import graphviz # graphviz is a package that helps visuali
```

Materials Used Layer by Layer

Left Contact (Back)

Electrical Properties

- Thermionic emission / surface recombination velocity (cm/s):
 - electrons: 1.00E+5
 - holes: 1.00E+7
 - Metal work function (eV): 5.0000
- Majority carrier barrier height:
 - relative to EF: 0.3700
 - relative to EV or EC: 0.3079

Layer 1 (HTL)

Absorption Interpolation Model

- thickness(μm): 0.150
- bandgap (eV): 2.170
- electron affinity (eV): 3.200
- dielectric permittivity (relative): 7.110
- CB effective density of state ($1/\text{cm}^3$): 2.020E+17
- VB effective density of state ($1/\text{cm}^3$): 1.100E+19
- electron thermal velocity (cm/s): 1.000E+7
- hole thermal velocity (cm/s): 1.000E+7
- electron mobility (cm^2/Vs): 2.000E+2
- hole mobility (cm^2/Vs): 8.000E+1
- no ND grading (uniform)
shallow uniform donor density ND ($1/\text{cm}^3$): 0.000E+0
- no NA grading (uniform)
shallow uniform acceptor density NA ($1/\text{cm}^3$): 1.000E+18

Recombination Model

- Band to band recombination
 - Radiative recombination coefficient (cm^3/s): 2.300E-9
 - Auger electron capture coefficient (cm^6/s): 0.000E+0
 - Auger hole capture coefficient (cm^6/s): 0.000E+0
- Recombination at defects: Summary
no defect added

Interface 1/2

Name of interface: Cu₂O / MASnI₃

Interface States:

- Defect 1
 - charge type: neutral
 - concentration: $N = 1.00E+10 / \text{cm}^2$
 - energy distribution: single; $E_t = 0.60 \text{ eV}$ above highest EV
 - this defect only, if active: $S_n(\text{left}) = 1.0E-2 \text{ cm/s}$, $S_n(\text{right}) = 1.0E-2 \text{ cm/s}$
 - this defect only, if active: $S_p(\text{left}) = 1.0E-2 \text{ cm/s}$, $S_p(\text{right}) = 1.0E-2 \text{ cm/s}$

Layer 2 (Absorber)

Absorption Interpolation Model

- thickness(μm): 0.350
- bandgap (eV): 1.5-1.7
- electron affinity (eV): 4.15-4.30
- dielectric permittivity (relative): 7-11
- CB effective density of state ($1/\text{cm}^3$): $2 \cdot 10E+18$
- VB effective density of state ($1/\text{cm}^3$): $2 \cdot 10E+18$
- electron thermal velocity (cm/s): $1.000E+7$
- hole thermal velocity (cm/s): $1.000E+7$
- electron mobility (cm^2/Vs): 20-100
- hole mobility (cm^2/Vs): 20-100
- no ND grading (uniform)
shallow uniform donor density ND ($1/\text{cm}^3$): $1.000E+9$
- no NA grading (uniform)
shallow uniform acceptor density NA ($1/\text{cm}^3$): $1.000E+9$

Recombination Model

- Band to band recombination
 - Radiative recombination coefficient (cm^3/s): $2.300E-9$
 - Auger electron capture coefficient (cm^6/s): $0.000E+0$
 - Auger hole capture coefficient (cm^6/s): $0.000E+0$
- Recombination at defects: Summary
 - Defect 1
 - charge type: neutral
 - total density ($1/\text{cm}^3$): Uniform $1.000E+16$
 - grading $N_t(y)$: uniform
 - energy distribution: gauss; $E_t = 0.60 \text{ eV}$ above EV; $E_{\text{kar}} = 0.10 \text{ eV}$
 - this defect only, if active: $\tau_n = 1.0E+1 \text{ ns}$, $\tau_p = 1.0E+1 \text{ ns}$
 - this defect only, if active: $L_n = 2.0E-1 \mu\text{m}$, $L_p = 2.0E-1 \mu\text{m}$

Interface 2/3

Name of interface: MASnI3 / TiO2

Interface States:

- Defect 1
 - charge type: neutral
 - concentration: $N = 1.00E+10 / \text{cm}^2$
 - energy distribution: single; $E_t = 0.60 \text{ eV}$ above highest EV
 - this defect only, if active: $\text{Sn}(\text{left}) = 1.0E-2 \text{ cm/s}$, $\text{Sn}(\text{right}) = 1.0E-2 \text{ cm/s}$
 - this defect only, if active: $\text{Sp}(\text{left}) = 1.0E-2 \text{ cm/s}$, $\text{Sp}(\text{right}) = 1.0E-2 \text{ cm/s}$

Layer 3 (ETL)

Absorption Interpolation Model

- thickness(μm): 0.150
- bandgap (eV): 3.200
- electron affinity (eV): 4.260
- dielectric permittivity (relative): 9.000
- CB effective density of state ($1/\text{cm}^3$): $2.200E+18$
- VB effective density of state ($1/\text{cm}^3$): $1.800E+18$
- electron thermal velocity (cm/s): $1.000E+7$
- hole thermal velocity (cm/s): $1.000E+7$
- electron mobility (cm^2/Vs): $2.000E+1$
- hole mobility (cm^2/Vs): $1.000E+1$
- no ND grading (uniform)
shallow uniform donor density ND ($1/\text{cm}^3$): $1.000E+16$
- no NA grading (uniform)
shallow uniform acceptor density NA ($1/\text{cm}^3$): $0.000E+0$

Recombination Model

- Band to band recombination
 - Radiative recombination coefficient (cm^3/s): $2.300E-9$
 - Auger electron capture coefficient (cm^6/s): $0.000E+0$
 - Auger hole capture coefficient (cm^6/s): $0.000E+0$
- Recombination at defects: Summary
no defect added

Right Contact (Front)

Electrical Properties

- Thermionic emission / surface recombination velocity (cm/s):
 - electrons: $1.00E+7$
 - holes: $1.00E+5$
 - Metal work function (eV): 4.4000
- Majority carrier barrier height:

- relative to EF: 0.1400
- relative to EV or EC: 0.0003

Importing the Dataset

Throughout this book, we will see many objects in the code named as `df`, which stands for dataframe, the primary data structure from the `pandas` package. It is similar to a spreadsheet or a table.

```
In [6]: df = pd.read_csv("63.5k-Unique-Devices.csv")
```

Read our dataset

```
In [7]: # Read our dataset
df
```

```
Out[7]:
```

	VB DOS	CB DOS	EA	Eg	e Mobility	h Mobility	Permittivity	Voc (V)	J: (mA/cm ²)
0	2.000000e+18	2.000000e+18	4.15	1.5	20	20	7	0.989106	15.32717
1	2.000000e+18	2.000000e+18	4.15	1.5	20	20	8	0.989203	15.33568
2	2.000000e+18	2.000000e+18	4.15	1.5	20	20	9	0.989295	15.34325
3	2.000000e+18	2.000000e+18	4.15	1.5	20	20	10	0.989382	15.34984
4	2.000000e+18	2.000000e+18	4.15	1.5	20	20	11	0.989464	15.35550
...	...	...	...	...	...	...	...	...	...
63495	1.000000e+19	1.000000e+19	4.30	1.7	100	100	7	1.043485	11.24508
63496	1.000000e+19	1.000000e+19	4.30	1.7	100	100	8	1.045393	11.25310
63497	1.000000e+19	1.000000e+19	4.30	1.7	100	100	9	1.047368	11.26060
63498	1.000000e+19	1.000000e+19	4.30	1.7	100	100	10	1.049370	11.26746
63499	1.000000e+19	1.000000e+19	4.30	1.7	100	100	11	1.051375	11.27359

63500 rows × 11 columns

Splitting Targets and Features

The original data is a combination of features (input parameters) and target variables (outputs) in one single dataframe, which isn't ideal. To make our next steps easier, We will split it into two dataframes:

1. `y` : target values (outputs)
2. `x` : features (inputs)

```
In [8]: #split df into two dataframes
x = df[['VB DOS', 'CB DOS', 'EA', 'Eg', 'e Mobility', 'h Mobility', 'Permittivity']]
y = df['PCE (%)']
```


Now, let's take a look at our target values first.

```
In [9]: y # our original dataset only with index and outputs
```

```
Out[9]: 0      11.7929
1      11.7865
2      11.7802
3      11.7739
4      11.7677
...
63495    9.1986
63496    9.2009
63497    9.2031
63498    9.2051
63499    9.2070
Name: PCE (%), Length: 63500, dtype: float64
```

Let's also take a look at the features generated.

```
In [10]: x #generated features
```

```
Out[10]:
```

	VB DOS	CB DOS	EA	Eg	e Mobility	h Mobility	Permittivity
0	2.000000e+18	2.000000e+18	4.15	1.5	20	20	7
1	2.000000e+18	2.000000e+18	4.15	1.5	20	20	8
2	2.000000e+18	2.000000e+18	4.15	1.5	20	20	9
3	2.000000e+18	2.000000e+18	4.15	1.5	20	20	10
4	2.000000e+18	2.000000e+18	4.15	1.5	20	20	11
...	...	...	...	...	...	...	...
63495	1.000000e+19	1.000000e+19	4.30	1.7	100	100	7
63496	1.000000e+19	1.000000e+19	4.30	1.7	100	100	8
63497	1.000000e+19	1.000000e+19	4.30	1.7	100	100	9
63498	1.000000e+19	1.000000e+19	4.30	1.7	100	100	10
63499	1.000000e+19	1.000000e+19	4.30	1.7	100	100	11

63500 rows × 7 columns

Features and background explanations

Perspective

While there is much to talk about from a device physics perspective (for each parameter), I have given you the most fundamental and basic definition & relevance of each. One thing to ponder on is that these parameters are all materials properties unique to each layer, dependent on innate functionalities based on chemical composition, crystallographic orientation, and a few other items. Additionally, they are all intimately involved in this wonderful action of energy conversion from light - electricity -> all happening in nm scales. There is tremendous complexity happening in such small length scales - light absorption,

electron+hole pair generation, electric fields pulling each charge in each direction, charge movement, charge loss (by recombination), etc. At the end of the day, each parameter is terribly involved, and often-times, co-dependent on each other. I hope that with the progression of the project, we can (beyond the fundamental layer of assigning 'weights' to each parameter in terms of impact on ultimate efficiency) also be able to perhaps find correlations between the parameters, and thereby shed keen insight into some fundamental physics of parameter-interaction.

Features

1. **Bandgap (or electronic bandgap)** of a given layer gives is an intrinsic materials property unique to the chemical composition of that layer. It gives you a sense of how much energy from the electromagnetic spectrum is absorbed in that layer. Photons with energy equal or more than the bandgap can be absorbed in that layer, while photons with energy less than the bandgap cannot pass through. For the ETL, since it's the first layer that the light hits after passing into the device, should ideally have a high bandgap, so that a significant portion of the electromagnetic spectrum passes through and gets to the perovskite (absorber) layer. Remember that the perovskite layer does all the light absorption to create the electron+hole pairs. The most ideal bandgap the perovskite (absorber layer) can have is 1.4 eV. The bandgap of the HTL isn't as important. As a fun fact, note that in the device, band edges (conduction and valence) of each layer with respect to each other is critical to ensure the directionality of electron flow and hole flow is maintained. The bandgap (offset between the conduction and valence bands) is a function of these band edges; however, it has its own physical meaning as well (as described above).
1. **Electron affinity:** It is the energy required to take the electron from the conduction band to the vacuum level (basically a level where the electron escapes the material). Again, this is a materials property, and each layer has its unique electron affinity. Electron affinity is more relevant to talk about for the perovskite (absorber) layer. Once the electron+hole pairs are created, they need to be conducted away to the ETL and HTL to ensure that an electric current is generated. The higher the value of electron affinity, the higher the barrier to take the electron from the absorber layer to the ETL. So, ideally, the perovskite layer should have a lower value for the electron affinity.
1. **Relative permittivity (also known as the dielectric constant)** gives you a sense of how easily a material polarizes under the application of an electric field. Given the fact that light is nothing but a fluctuating interaction of electric and magnetic fields, the higher the relative permittivity, the more it is prone to create electron+hole pairs. The relative permittivity of the perovskite (absorber) material is therefore more relevant to our case; the higher the value, the better.
1. **Conduction band density of states** - basically the higher the number in the perovskite and the ETL, the better. As the name suggests, this is a materials property showcasing the ability to take on and conduct more electrons in the conduction band. Remember, once electron+hole pairs are created in the perovskite, the electrons will conduct from the conduction band of the perovskite to the conduction band of the ETL. Therefore, the

higher the conduction band density of states of perovskite and ETL, the better. We don't have much of a dependency on the conduction band density of states of the HTL here.

1. ****The flip of bullet 4 is valence band density of states.**** Once electron+hole pairs are created in the perovskite, the holes will conduct from the valence band of the perovskite to the valence band of the HTL. Therefore, the higher the valence band density of states of perovskite and HTL, the better. We don't have much of a dependency on the valence band density of states of the ETL here.

1. **Electron mobility** - as is obvious, we want this to be higher in the perovskite and the ETL. Once the electron+hole pairs are created, the sole objective is to get the electrons out of the device (flow from perovskite -> ETL -> electrode) as effectively as possible without incurring losses (ways in which the electron might recombine with holes that exist in the material, thereby getting 'lost'). We don't have much of a dependency on the electron mobility in the HTL here.

1. **Hole mobility** - same logic as #6 above. In this case, we want hole mobility to be as high as possible in the perovskite and the HTL. We don't have much of a dependency on the hole mobility of the ETL here.

ETL, HTL and Perovskite thickness values are constants after being optimized to give maximum device efficiency.

Correlation Matrix

```
In [11]: corr = df.drop(columns=['Voc (V)', 'Jsc (mA/cm2)', 'FF (%)']).corr()
corr.iloc[:,7].sort_values(ascending=False)
```

```
Out[11]: h Mobility      0.302746
e Mobility      0.000688
Permittivity    0.000307
VB DOS         -0.068954
CB DOS         -0.104182
EA             -0.149510
Eg            -0.868535
Name: PCE (%), dtype: float64
```

```
In [12]: corr = corr.abs()
cor = corr.iloc[:,7].sort_values(ascending=False)
```

```
In [13]: cor.columns = ['p', 'n']
print(cor)

Eg            0.868535
h Mobility     0.302746
EA            0.149510
CB DOS        0.104182
VB DOS        0.068954
e Mobility     0.000688
Permittivity  0.000307
Name: PCE (%), dtype: float64
```

Setting Up Model Evaluation

Before jumping in to model building, we have to think about: How are we going to know how well our models are performing?

To get an unbiased estimate of model error, we can employ a simple cross-validation technique: Setting aside a random subset of data as our test set, which the model won't access during the training process. The rest of the data constitutes the training set, which is used to train the model. This allows us to then evaluate model performance by comparing model prediction on both seen data (training set) and unseen data (test set).

Establishing Train/Test Split

For now let's split off 25% of the data for testing. We use the `train_test_split()` method from the `scikit-learn` package to generate the split. In this case, our input data `X` and output data `y` are split into 4 parts:

- `x_train` : training set input data
- `x_test` : test set input data
- `y_train` : training set output data
- `y_test` : test set output data

```
In [14]: # Generate train/test split by reserving 25% of data as test set
test_fraction = 0.25
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=test_fr
```

Fitting the Decision Tree Model

We'll train or "fit" a decision tree model using the `RandomForestRegressor().fit()` class from the `scikit-learn` package we imported at the beginning of the notebook. The actual class name that we're using is called `RandomForestRegressor`. This `RandomForestRegressor` class can be configured to be identical to a decision tree, however the reason we'll use it is because later on it can give us more flexibility if we need to change the model. A decision tree model is basically a random forest model with 1 tree, so we'll set the `n_estimators` hyperparameter to 1 to make the model mimic a single decision tree.

```
In [15]: # fit the decision tree model
Default_model = RandomForestRegressor(random_state=1, n_estimators=1, bootstr
print('Model training complete.')
print('Tree depth:', [estimator.tree_.max_depth for estimator in Default_mo
for importance in zip (estimator.feature_importances_ for estimator in Defa
print (importance)
#print('Leaf nodes:', [estimator.tree_.n_leaves for estimator in Default_mo
```

Model training complete.

Tree depth: [22]

```
(array([1.31313211e-02, 1.25889886e-02, 9.70352345e-02, 7.74029153e-01,
        8.97543978e-07, 1.03162450e-01, 5.19553511e-05]),)
```

Evaluating Model Performance

To evaluate the model performance, we will use it to predict outputs of materials it was trained on (training set) as well as materials it has not seen before (test set), and we will compare its performance on both datasets, and the accuracy of test predictions.

```
In [16]: Train_predictions = Default_model.predict(x_train)
# Make predictions on training data
Test_predictions = Default_model.predict(x_test)
# Make predictions on testing data
```

```
In [17]: def rmse(y_true, Y_predictions): return round(np.sqrt(metrics.mean_squared_
def rmse_std(y_true, Y_predictions): return round(rmse(y_true, Y_prediction
def mae(y_true, Y_predictions): return round(metrics.mean_absolute_error(y_
def r2(y_true, Y_predictions): return round(metrics.r2_score(y_true, Y_prec
```

```
In [18]: def parity_stats(y_left_true, Y_left_predictions, y_right_true, Y_right_predic

rmse_left = rmse(y_left_true, Y_left_predictions)
rmse_std_left = rmse_std(y_left_true, Y_left_predictions)
mae_left = mae(y_left_true, Y_left_predictions)
r2_left = r2(y_left_true, Y_left_predictions)

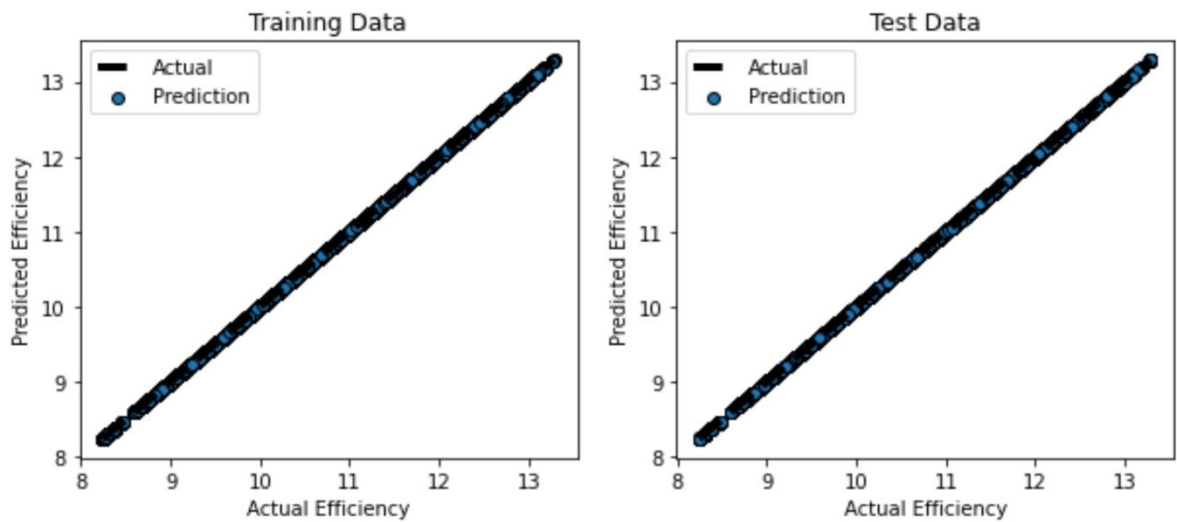
rmse_right = rmse(y_right_true, Y_right_predictions)
rmse_std_right = rmse_std(y_right_true, Y_right_predictions)
mae_right = mae(y_right_true, Y_right_predictions)
r2_right = r2(y_right_true, Y_right_predictions)

stats_df = pd.DataFrame({'Error Metric' : ['RMSE', 'RMSE/std', 'MAE', '
title_left : [str(rmse_left), rmse_std_left, s
title_right: [str(rmse_right), rmse_std_right,
'Note' : ['(0.0 for perfect prediction)', '(0.

return stats_df
```

```
In [19]: def parity_plots(y_left_true, Y_left_predictions, y_right_true, Y_right_predic
title_left="Training", title_right="Test"):
fig1, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.scatter(y_left_true, Y_left_predictions, edgecolors=(0, 0, 0))
ax1.plot([y_left_true.min(), y_left_true.max()], [y_left_true.min(), y_
ax1.legend(["Actual", "Prediction"])
ax1.set_xlabel('Actual Efficiency')
ax1.set_ylabel('Predicted Efficiency')
ax1.set_title(title_left)
ax2.scatter(y_right_true, Y_right_predictions, edgecolors=(0, 0, 0))
ax2.plot([y_right_true.min(), y_right_true.max()], [y_right_true.min(),
ax2.legend(["Actual", "Prediction"])
ax2.set_xlabel('Actual Efficiency')
ax2.set_ylabel('Predicted Efficiency')
ax2.set_title(title_right)
plt.savefig("parity.tiff", dpi=300)
plt.show()
```

```
In [20]: parity_plots(y_train, Train_predictions, y_test, Test_predictions,
title_left="Training Data", title_right="Test Data
parity_stats(y_train, Train_predictions, y_test, Test_predictions, "Training Da
```



Out[20]:	Error Metric	Training Data	Test Data	Note
0	RMSE	0.0	0.001	(0.0 for perfect prediction)
1	RMSE/std	0.0	0.001	(0.0 for perfect prediction)
2	MAE	0.0	0.0002	(0.0 for perfect prediction)
3	R2	1.0	1.0	(1.0 for perfect prediction)

```
In [21]: # Accuracy train data Default_model
Default_model.score(x_train, y_train)
```

Out[21]: 0.999999999997134

```
In [22]: # Accuracy test data Default_model
Default_model.score(x_test, y_test)
```

Out[22]: 0.9999991050381218

```
In [23]: # print the average performance of models with the best hyperparameter during
def CV_best_stats(CV, y_true):
    idx = CV.best_index_

    rmse = round(np.sqrt(-CV.cv_results_['mean_test_neg_mean_squared_error'][idx]), 4)
    rmse_std = round(rmse/np.std(y_true), 4)
    mae = round(-CV.cv_results_['mean_test_neg_mean_absolute_error'][idx], 4)
    r2 = round(CV.cv_results_['mean_test_r2'][idx], 4)

    print("Average test RMSE: ", rmse, "(0.0 for perfect prediction)")
    print("Average test RMSE/std: ", rmse_std, "(0.0 for perfect prediction)")
    print("Average test MAE: ", mae, "(0.0 for perfect prediction)")
    print("Average test R2: ", r2, "(1.0 for perfect prediction)")
    return rmse, rmse_std, mae, r2
```

```
In [24]: default_dict = {'n_estimators':[1]}
default_CV = GridSearchCV(Default_model, default_dict, return_train_score=True,
                           scoring=['neg_mean_squared_error', 'r2', 'neg_mean_absolute_error'],
                           refit='neg_mean_squared_error')
default_CV = default_CV.fit(x_train,y_train)
default_CV_stats = CV_best_stats(default_CV,y_train)
```

```
Average test RMSE:  0.0014 (0.0 for perfect prediction)
Average test RMSE/std:  0.0013 (0.0 for perfect prediction)
Average test MAE:  0.0004 (0.0 for perfect prediction)
Average test R2:  1.0 (1.0 for perfect prediction)
```

```
In [25]: kfold = RepeatedKFold(n_splits=10, random_state=1, n_repeats=10)
CV = GridSearchCV(Default_model, default_dict, cv=kfold, return_train_score=True,
                  scoring=['neg_mean_squared_error', 'r2', 'neg_mean_absolute_error'],
                  refit='neg_mean_squared_error')
CV = CV.fit(x_train,y_train)
```

```
In [26]: # Extract cross validation performance metrics for the optimized model
kfold_CV_stats = CV_best_stats(CV,y_train)
```

```
Average test RMSE:  0.0011 (0.0 for perfect prediction)
Average test RMSE/std:  0.001 (0.0 for perfect prediction)
Average test MAE:  0.0003 (0.0 for perfect prediction)
Average test R2:  1.0 (1.0 for perfect prediction)
```

Relative Importance of Features

These methods are most often used for prediction, moreover, looking at the feature importances can give us a sense of which of your variables have the most effect in these models. We can:

1. use that information to engineer new features,
2. drop out features that look like noise,
3. inform us as we continue building models.

Decision trees, and the ensemble methods that are built on them, work by splitting the data into subsets which most heavily belong to one class. The tree will continue to build different subsets until it understands and represents the relationship of the variables with the target. All kinds of tree methods calculate splits by mathematically determining which split will most effectively help distinguish the classes. Because that is their method, the sklearn instances of these models have a `feature_importances_` attribute, which returns an array of each feature's importance in determining the split.

```
In [27]: im = pd.DataFrame({'Features':x_train.columns, 'Importances':Default_model.feature_importances_})
im
```

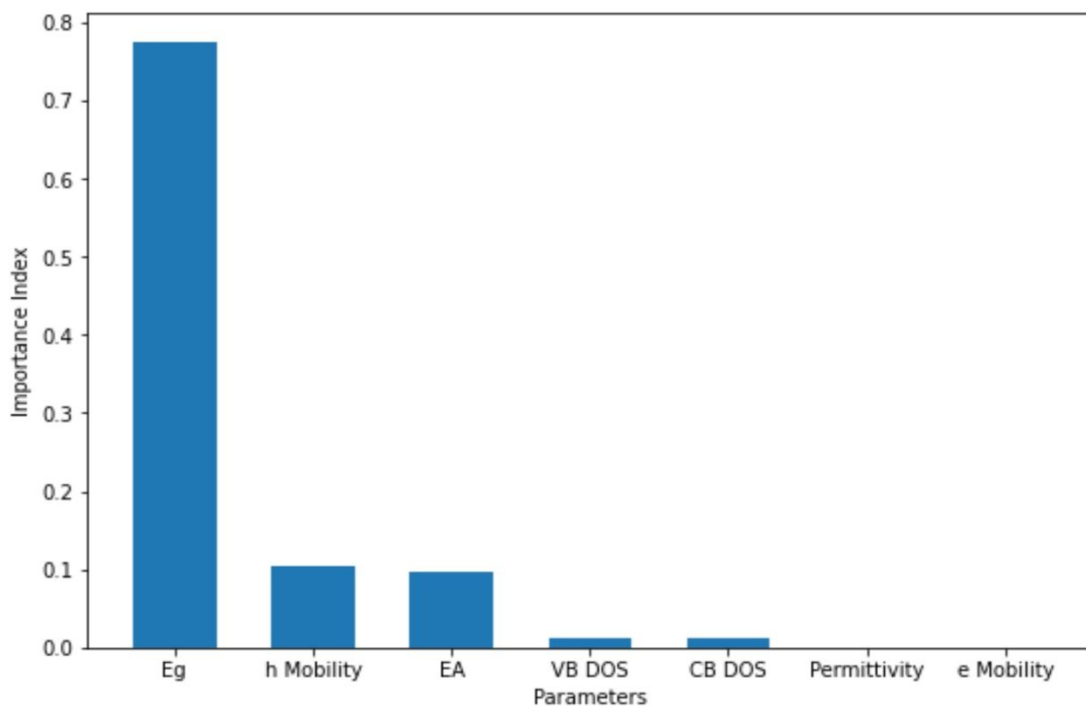
```
Out[27]:
```

	Features	Importances
3	Eg	7.740292e-01
5	h Mobility	1.031625e-01

	Features	Importances
2	EA	9.703523e-02
0	VB DOS	1.313132e-02
1	CB DOS	1.258899e-02
6	Permittivity	5.195535e-05

Feature importance: The higher, the more important the feature. See the cell below:

```
In [28]: plt.figure(figsize=(9,6))
plt.bar(im['Features'], im['Importances'], width=0.6)
plt.xlabel('Parameters')
plt.ylabel('Importance Index')
plt.show
plt.savefig("FeatureImportance.tiff",dpi=300)
```



Dropping the Less Important Features

As we can see the above cell, CB DOS , e Mobility , Permittivity are less important compared to the other input parameters (features). So we are going to drop them for now and want to see how it affects our model performance.

```
In [ ]: # Dropping the Less Important Features
X = x.drop(columns=['CB DOS', 'e Mobility', 'Permittivity', 'VB DOS'])
```

```
In [ ]: new_df = pd.concat([X, df.iloc[:,10]], axis=1)
new_df
```

```
In [33]: new_df_clean = new_df.groupby(['EA', 'h Mobility', 'Eg']).agg({'PCE (%)': ['min', 'max']})
new_df_clean.columns = ['PCE_min', 'PCE_max']
new_df_clean = new_df_clean.reset_index()
```

```
In [34]: new_df_clean
```

```
Out[34]:
```

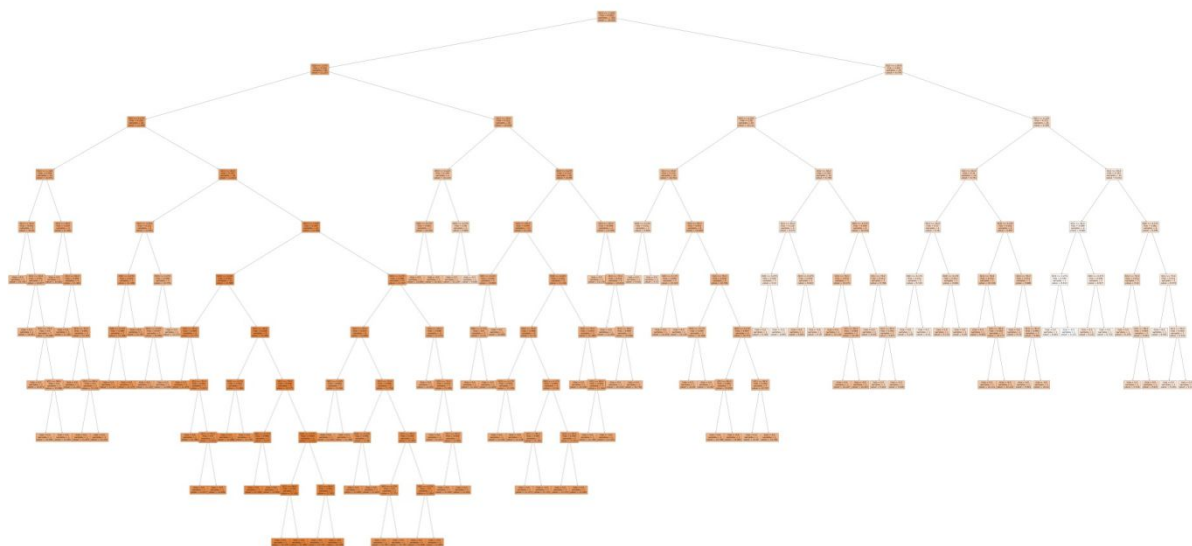
	EA	h Mobility	Eg	PCE_min
0	4.15	20	1.50	10.7453
1	4.15	20	1.55	10.8041
2	4.15	20	1.60	10.6065
3	4.15	20	1.65	9.9321
4	4.15	20	1.70	9.2289
...	...	...	...	...
95	4.30	100	1.50	12.2275
96	4.30	100	1.55	11.5252
97	4.30	100	1.60	10.7575
98	4.30	100	1.65	9.9275
99	4.30	100	1.70	9.1964

100 rows × 4 columns

```
In [35]: output_path = "new_df_clean.csv"
new_df_clean.to_csv(output_path)
```

```
In [36]: X = new_df_clean[['EA', 'h Mobility', 'Eg']]
Y = new_df_clean['PCE_min']
```

```
In [37]: Default_model_tree = RandomForestRegressor(random_state=1, n_estimators=1, bootstrap=True)
plt.figure(figsize=(200, 100))
tree.plot_tree(Default_model_tree.estimators_[0], filled=True)
plt.savefig("d-tree.pdf", dpi=600)
```



SHAP Analysis

The package shap is a package of methods most often used for prediction, moreover, looking at the relative impact of features can give us a sense of which of your variables have the most effect in these models. We can:

1. use that information to engineer new features,
2. drop out features that look like noise,
3. inform us as we continue building models.

In [12]: `import shap`

In [13]: `fig = plt.figure(figsize=(9,6))
explainer=shap.TreeExplainer(Default_model,x_test)
shap_values=explainer.shap_values(x_test)
shap.summary_plot(shap_values,x_test, show=False)
plt.savefig("rf.png",dpi=600, bbox_inches='tight')`

